



第五章 程序的执行

主要内容



华中科技大学

- ◆ 5.1 程序执行概述
- ◆ 5.2 流水线方式下指令的执行





5.1.程序执行概述

程序由指令组成，指令按顺序存放在连续的存储单元中。程序启动后，计算机能自动逐条取出程序中的指令并执行。



一、程序及指令的执行过程

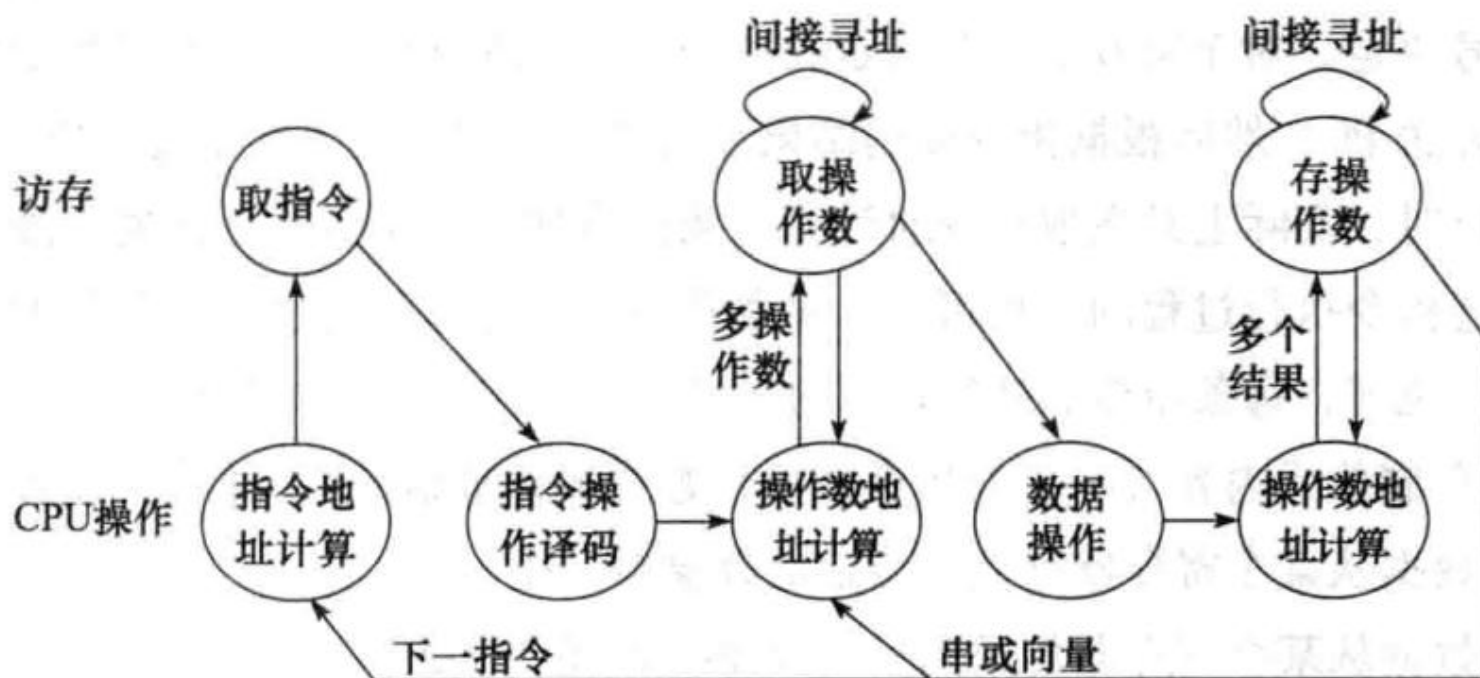
正常情况下，指令按其存放顺序执行。遇到需改变程序执行流程时，用相应的转移指令（包括无条件转移指令、条件转移指令、调用指令和返回指令等）来改变程序执行流程。

将要执行的指令所在存储单元的地址由程序计数器PC给出，通过改变PC的值来控制执行顺序。



一、程序及指令的执行过程

指令执行过程





一、程序及指令的执行过程

CPU执行一条指令的过程可以分为以下几个步骤：

- ①取指令。**
- ②指令译码**
- ③源操作数地址计算并取操作数。**
- ④执行数据操作**
- ⑤目的操作数地址计算并存结果。**
- ⑥下条指令地址计算并将其送PC。**



一、程序及指令的执行过程

①取指令。

从PC所指出的存储单元中取出指令送到指令寄存器(IR)

。

②指令译码

对IR中的指令操作码进行译码，根据不同的指令操作码译出不同的控制信号。



一、程序及指令的执行过程

③源操作数地址计算并取操作数。

根据寻址方式确定源操作数地址计算方式，若是存储器数据，则需要一次或多次访存；若是寄存器数据，则直接从寄存器取数后转到下一步进行数据操作。

④执行数据操作

在ALU或加法器等运算部件中对取出的操作数进行运算。





一、程序及指令的执行过程

⑤目的操作数地址计算并存结果。

根据寻址方式确定目的操作数地址计算方式，若是存储器数据，则需要一次或多次访存(间接寻址时);若是寄存器数据，则在进行数据操作时直接存结果到寄存器。

如果是串操作或向量运算指令，则可能会并行执行或循环执行第③~⑤步多次。



一、程序及指令的执行过程

⑥下条指令地址计算并将其送PC。

顺序执行时，下条指令地址的计算，只要将PC加上当前指令长度即可。如果译码的是转移类指令时，则需要根据条件标志、操作码和寻址方式等确定下条指令地址。



一、程序及指令的执行过程

对于上述过程的第①步和第②步，所有指令的操作都一样；而对于第③~⑤步，不同指令的操作可能不同，它们完全由第②步译码得到的控制信号控制。也即指令的功能由第②步译码得到的控制信号决定。对于第⑥步，若是定长指令字，处理器会在第①步取指令的同时计算出下条指令地址并送PC，然后根据指令译码结果和条件标志决定是否在第⑥步修改PC的值，因此，在顺序执行时，实际上是在取指令时计算下条指令地址，第⑥步什么也不做。



一、程序及指令的执行过程

总结上述指令执行过程，每条指令的功能总是通过对以下四种基本操作进行组合来实现：

- ①读取某存储单元内容(可能是指令或操作数或操作数地址)，并将其装入某个寄存器。
- ②把一个数据从某个寄存器存储到给定的存储单元中。
- ③把一个数据从某个寄存器传送到另一个寄存器或者ALU。
- ④在ALU中进行某种算术运算或逻辑运算，并将结果传送到某个寄存器。



主要内容



华中科技大学

- ◆ 5.1 程序执行概述
- ◆ 5.2 流水线方式下指令的执行





5.2.流水线方式下指令的执行

串行方式下，CPU在执行完一条指令后才取出下条指令执行。串行方式没有充分利用执行部件的并行性，因而指令执行效率低。

指令的执行可以采用流水线方式，即将多条指令的执行相互重叠起来，以提高CPU执行指令的效率。



一、流水线的基本原理

一条指令的执行过程可被分成若干个阶段。每个阶段都在相应的功能部件中完成。如果将各阶段看成相应的流水段，则指令的执行过程就构成了一条指令流水线。假定一条指令流水线由如下5个流水段组成：

取指令(IF):根据PC的值从存储器取出指令。

指令译码(ID):产生指令执行所需的控制信号。

取操作数(OF):读取存储器操作数或寄存器操作数。

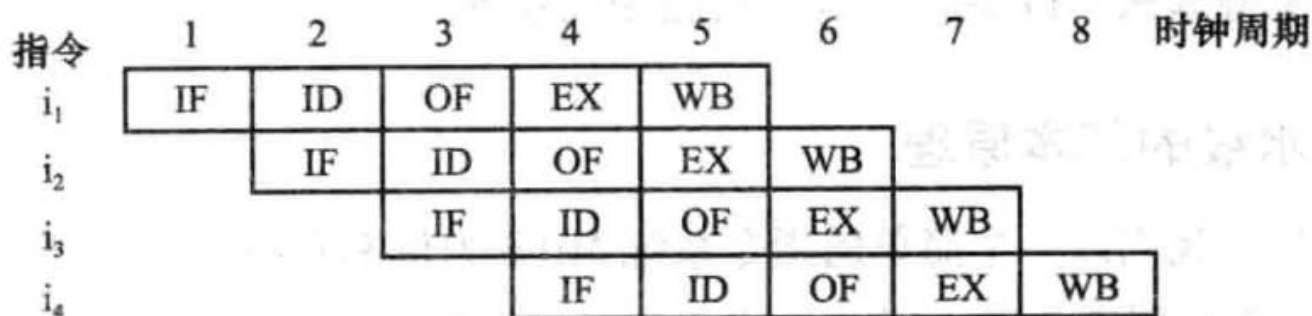
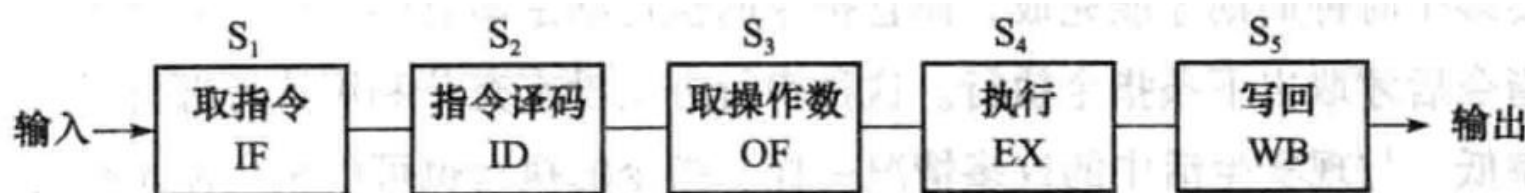
执行(EX):对操作数完成指定操作。

写回(WB):将操作结果写入存储器或寄存器。

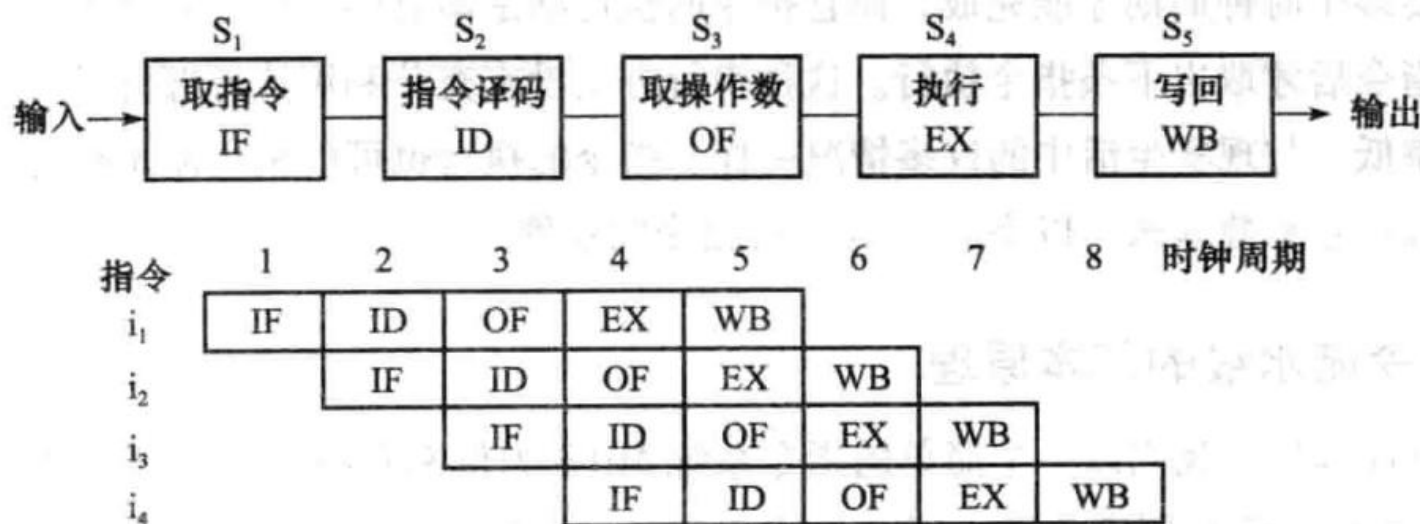


一、流水线的基本原理

进入流水线的指令流，由于后一条指令的第 i 步与前一条指令的第 $i+1$ 步同时进行，从而使一串指令总的完成时间大为缩短。



一、流水线的基本原理



如图所示，在理想状态下，完成4条指令的执行只用了8个时钟周期，若是非流水线方式的串行执行处理，则需要20个时钟周期。流水线方式充分利用了执行部件的并行性。



一、流水线的基本原理

例：一条指令需要用以下5个阶段完成操作：

- ①取指：200ps;
- ②译码和读操作数：50ps;
- ③ALU操作：100ps;
- ④读存储器：200ps;
- ⑤结果写寄存器：50ps。

这条指令的总执行时间为 $200 + 50 + 100 + 200 + 50 = 600\text{ps}$ 。





一、流水线的基本原理

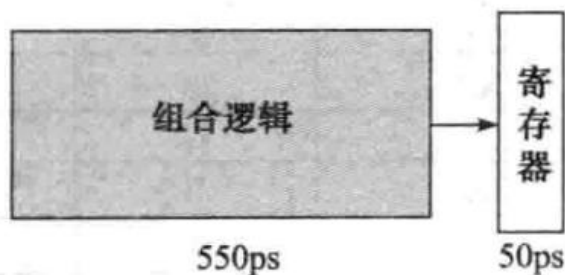
在串行执行情况下，指令不做切分，因此要求所有指令都在一个时钟周期内完成。由于系统的时钟周期是固定的，因此时钟周期需要设置为最复杂指令的延时。设上述指令为最复杂指令，则时钟周期必须设为600ps。

系统的指令吞吐率为 $1/(600 \times 10^{-12}) = 1.67\text{GIPS}$ 。



一、流水线的基本原理

串行方式指令执行过程



CPI=1, 指令延时为600ps
指令吞吐率为1.67GIPS

指令串行执行, 程序执行时间为: 指令条数 $\times$ 600ps



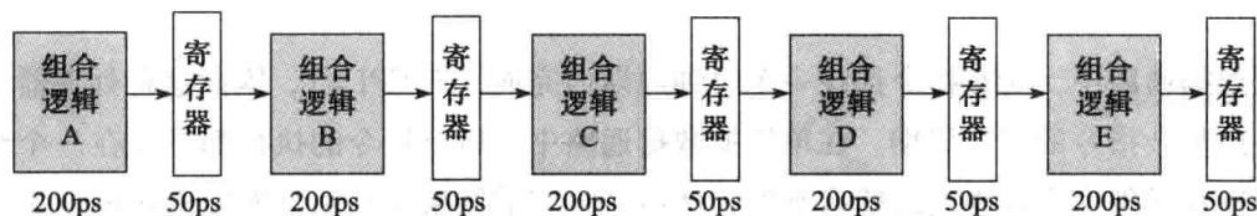
一、流水线的基本原理

作为比较，在流水线方式下，用最复杂指令的功能段个数，作为流水线段个数；用最复杂共功能段的操作用时，作为流水段的时间长度，也是系统的时钟周期。在流水线方式下，每个阶段还需要将阶段结果保存到寄存其中。设上述指令为最复杂指令，则流水线段数为5，时钟周期为 $200 + 50 = 250\text{ps}$ 。

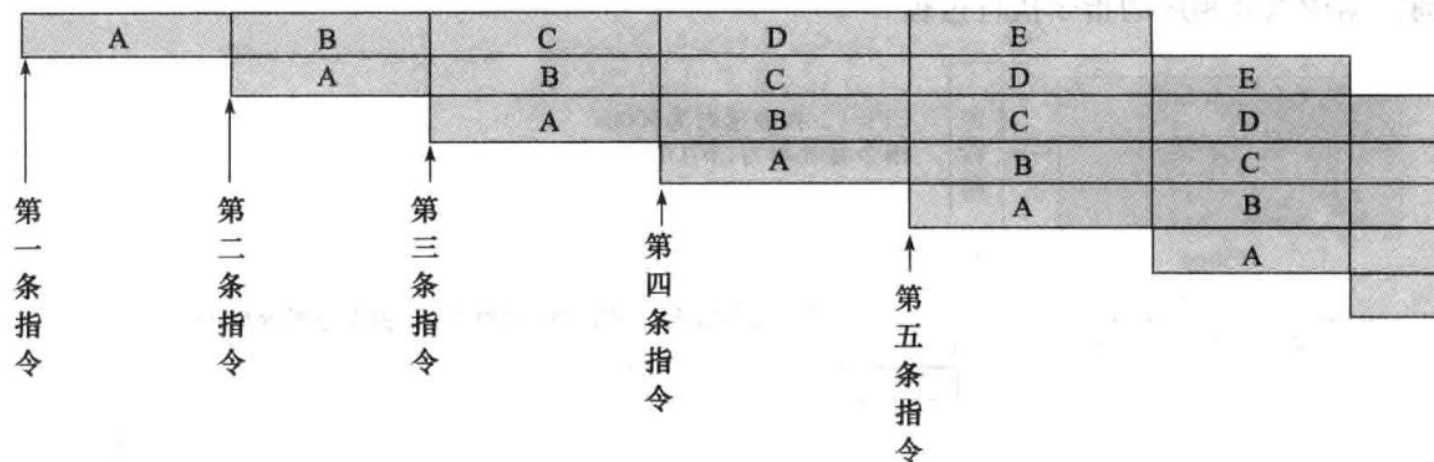


一、流水线的基本原理

流水线方式指令执行过程



CPI=1, 指令延时为1.25ns, 指令吞吐率为4GIPS





一、流水线的基本原理

上述指令的执行时间为5个时钟周期。

但在理想情况下，每个时钟周期有一条指令完成执行。指令吞吐率为 $1/(250 \times 10^{-12}) = 4\text{GIPS}$ ，吞吐率是串行方式的 $4/1.67 = 2.4$ 倍。





二、CISC和RISC指令集

按流水线方式执行指令的关键是，将指令集中所有指令都分成相同数目的功能段，并让每个功能段的执行时间都相同。



二、CISC和RISC指令集

有利于流水线执行的指令集特征包括：

- 指令长度尽量一致，有利于简化取指令和指令译码操作
- 指令格式尽量规整，尽量保证源寄存器的位置相同。
- 采用load/store型指令风格。指令集中只有load指令和store指令能访问存储器，其他指令一律不能访问。可以保证除load和store指令外的其他指令在执行阶段都不访问存储器，有利于减少操作步骤，以规整流水线。
- 数据和指令在存储器中要“对齐”存放。有利于减少访存次数，使所需数据在一个流水段内就能从存储器中得到。



二、CISC和RISC指令集

按以上特征对指令集进行分类，有CISC和RISC两种不同的指令设计风格。

1.CISC风格指令系统

2.RISC风格指令集





1.CISC风格指令系统

CISC指令系统设计的主要特点如下。

- ①指令系统复杂。指令条数多，寻址方式多，指令格式多而复杂，指令长度可变，操作码长度可变。**
- ②指令周期长。绝大多数指令需要多个时钟周期才能完成。**
- ③相关指令会产生显式的条件码，存放在专门的标志寄存器(或称状态寄存器)中，可用于条件转移和条件传送等指令。**
- ④指令周期差距大。各种指令都能访问存储器，有些指令还需要多次访问存储器，使得简单指令和复杂指令所用的时钟周期数相差很大，不利于指令流水线的实现。**
- ⑤难以进行编译优化。由于编译器可选指令序列增多，使得目标代码组合增加，从而增加了目标代码优化的难度。**



1.CISC风格指令系统

例：

VAX11/780指令系统包含了16种寻址方式、9种数据格式、303条指令，而且一条指令包含1~2个字节的操作码和后续N个操作数说明符，而一个操作数说明符的长度可达1~10个字节。

复杂指令系统使得实现越来越复杂，不仅增加了研制周期和成本，而且难以保证正确性，甚至因为指令太复杂而无法采用硬连线路控制器，只能采用慢速的微程序控制器，从而降低了系统性能。



2.RISC风格指令集



华中科技大学

对CISC程序调查分析表明，各种指令的使用频率相当悬殊。

	简单指令	复杂指令
使用频率	常使用	不常使用
占指令系统	20%	80%
占程序代码	80%	20%

杂指令在程序中的出现频率却很低，造成了硬件资源的大量浪费。



2.RISC风格指令集



华中科技大学

1975年IBM公司开始研究指令系统的合理性问题，John Cocke领导的一个研究小组提出了精简指令集计算机(Reduced Instruction Set Computer,简称RISC)的概念。

RISC通过简化指令系统使计算机结构更加简单合理，从而提高机器的性能。



2.RISC风格指令集



华中科技大学

RISC指令系统的主要特点包括：

- ①指令数目少**
- ②指令格式规整**
- ③采用load/store型指令设计风格**
- ④采用流水线方式执行指令**
- ⑤采用大量通用寄存器**
- ⑥采用硬连线路控制器**
- ⑦实现细节对机器级程序可见**





2.RISC风格指令集

- ①指令数目少。只包含使用频度高的简单指令。
- ②指令格式规整。寻址方式少，指令格式少，指令长度一致，指令中操作码和寄存器编号等位置固定，便于取指令、指令译码以及提前读取寄存器内容等。
- ③采用load/store型指令设计风格。一条指令的执行阶段最多只有一次存储器访问操作。

2.RISC风格指令集



华中科技大学

④采用流水线方式执行指令。规整的指令格式有利于采用流水线方式执行，除load/store指令外，其他指令都只需一个或小于一个时钟周期就可完成，指令周期短。

⑤采用大量通用寄存器。编译器可将更多的局部变量分配到寄存器中，并且在过程调用时通过寄存器进行参数传递而不是通过栈进行传递，以减少访存次数。

⑥采用硬连线路控制器。指令少而规整使得控制器的实现变得简单，可以不用或少用微程序控制器。



2.RISC风格指令集



华中科技大学

⑦实现细节对机器级程序可见。例如，有些RISC机器禁止一些特殊的指令序列，有些则规定条件转移指令后面必须填充若干条必须执行的指令等，这些都给编译器的设计和优化设定了相应的约束条件。





2.RISC风格指令集

RISC指令系统简单，所以：

- **CPU的控制逻辑简化**
- **芯片上有更多的通用寄存器**
- **可以采用速度较快的硬连线控制器**
- **更适合于采用指令流水技术**

这些都可以使指令的执行速度进一步提高。

指令数量少，固然使编译工作量加大，但由于指令系统中的指令都是精选的，编译时间少，反过来对编译程序的优化也是有利的。



2.RISC风格指令集



华中科技大学

例：

20世纪80年代中期，RISC技术蓬勃发展并广泛使用，先后出现了PowerPC、MIPS、Sun SPARC、Compaq Alpha等高性能RISC芯片以及相应的计算机。





2.RISC风格指令集

例：像IA-32这种指令格式不规整、寻址方式复杂多样、指令功能强弱不均的指令集来说，用流水线方式来实现比较困难。同时Intel为了保持市场占有率，必须保持指令系统的兼容，因而不能改变其指令系统设计风格，但流水线方式执行指令肯定比串行执行方式好，那该怎么办呢？

Intel采用了一种称为“CISC壳、RISC核”的设计策略，IA-32架构在指令集体系结构(ISA)层面是CISC风格的指令系统，但是在执行指令的微体系结构层面采用的是RISC风格的微操作流水线执行机制。它将一条复杂的CISC风格的机器指令分解为多个RISC风格的微操作，对于每个微操作再用流水线方式来实现。



2.RISC风格指令集



华中科技大学

例：

随着后PC时代的到来，个人移动设备的使用和嵌入式系统的应用越来越广泛，像ARM处理器等这些采用RISC技术的产品又迎来了新的机遇，目前在嵌入式系统中占有绝对优势。

